



Stellenbosch

UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

The Compression of Deep Reinforcement Learning Agents onto Embedded Microcontrollers

Mechanical Project 478
Final Report

AL Beukes
24960098

Supervisor
Prof HA Engelbrecht

2026

Plagiarism declaration

I have read and understand the Stellenbosch University Policy on Plagiarism and the definitions of plagiarism and self-plagiarism contained in the Policy [Plagiarism: The use of the ideas or material of others without acknowledgement, or the re-use of one's own previously evaluated or published material without acknowledgement or indication thereof (self-plagiarism or text-recycling)].

I also understand that direct translations are plagiarism, unless accompanied by an appropriate acknowledgement of the source. I also know that verbatim copy that has not been explicitly indicated as such, is plagiarism.

I know that plagiarism is a punishable offence and may be referred to the University's Central Disciplinary Committee (CDC) who has the authority to expel me for such an offence.

I know that plagiarism is harmful for the academic environment and that it has a negative impact on any profession.

Accordingly, all quotations and contributions from any source whatsoever (including the internet) have been cited fully (acknowledged); further, all verbatim copies have been expressly indicated as such (e.g. through quotation marks) and the sources are cited fully.

I declare that, except where a source has been cited, the work contained in this assignment is my own work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.

I declare that have not allowed, and will not allow, anyone to use my work (in paper, graphics, electronic, verbal or any other format) with the intention of passing it off as his/her own work.

I know that a mark of zero may be awarded to assignments with plagiarism and that no opportunity be given to submit an improved assignment.

Signature:



.....

Name: Achaia Llew Beukes

Student no: 24960098

Date: 24/07/2026

AI Use Declaration

1. I am convinced and can support my claim that my assessment product is an indication of my own learning, knowledge, skills, and understanding.
2. Where I have used AI tools for enhancing my own creation of ideas and words, I acknowledge that I have to declare it.
3. Where I have used AI tools for generating new ideas, words, code, image-prompts for other AI Image-generating tools, or structure and even presentations (or other AI tools that can be used as assistants to the knowledge building and representing process), I have declared and documented the use of such tools and I am prepared to talk about the process I used and what it contributed to my learning and insights.
4. I am aware that the lecturer can ask me to demonstrate my learning, for example through explaining the choices I made in terms of approach, content used, literature selected, conclusions drawn, etc. through an additional assessment like an oral (for example).
5. I understand that if I am not able to agree to the above points, there is a chance that my academic behaviour will be deemed unethical and might lead to a disciplinary case being brought against me on the grounds of cheating or plagiarism and that the standard procedures for such behaviour will be followed.
6. As per the Disciplinary Code of SU (par. 10.2.1 and 10.2.2), I understand that I take responsibility for the integrity of my work, which includes the obligation to ask for clarification from an academic member of staff if I am unsure of anything, and that I strictly adhered to all instructions received in the course of the academic assessment by relevant and authorized staff (whether the instruction is in oral or written format).
7. I understand that when I am not able to document and declare my use of AI tools, this behaviour will be deemed as cheating in examinations and assessments (Disciplinary Code 1.1 c.) as I referred to “unauthorized notes, books, electronic devices or other reference material”.

Student Number: 24960098	Signature: 
Initials and Surname: AL Beukes	Date: 24 July 2026

Executive Summary

Title of Project
The Compression of Deep Reinforcement Learning Agents onto Embedded Microcontrollers
Objectives
To implement a DRL algorithm on a desktop computer and train the agent on images from a virtual environment until a baseline performance is established. Then, compress the policy network and deploy the agent onto a resource constrained embedded device.
What is current practice and what are its limitations?
The limitations are compressing Reinforcement Learning agents to deploy on extremely resource-constrained embedded devices without significant performance loss.
What is new in this project?
Multiple compression strategies will be evaluated against each other.
If the project is successful, how will it make a difference?
Most compression strategies have been implemented with supervised learning and therefore this research will make a difference in establishing a reliable method for compressing Deep Reinforcement Learning algorithms, of which there is limited research in.
What are the risks to the project being a success? Why is it expected to be successful?
The risk to the success of the project is failure to compress the policy network of the DRL algorithm sufficiently to deploy onto the embedded device.
What contributions have/will other students made/make?
No other contributions will be made by other students for the duration of this project.
Which aspects of the project will carry on after completion and why?
Testing the compression method in other environments. Therefore, further robust evaluation on model performance can be observed.
What arrangements have been/will be made to expedite continuation?
The entire project code will be uploaded to GitHub for continuation at project EOL.

Table of Contents

	Page
Plagiarism declaration	i
AI Use Declaration	ii
Table of Contents	iv
List of figures.....	v
List of tables	vi
1 Introduction	1
1.1 Background.....	1
1.2 Objectives	1
1.3 Motivation	2
2 Literature Review	3
2.1 Network Pruning and Fine-Tuning Strategies	3
2.2 LRP-Based Filter Pruning and Policy Distillation.....	3
2.3 Structured Pruning with Scaled Policy Constraints	4
3 Progress Completed.....	5
3.1 DRL Agent Development	5
3.1.1 Tabular Q-Learning Model	5
3.1.2 Translation to Deep Q-Network.....	6
3.1.3 DQN Model Architecture and Optimization.....	7
3.2 Policy Network Compression	8
3.2.1 Research Network Compression Methods	8
4 Planned Activities	9
4.1 Policy Network Compression	9
4.2 Embedded Deployment.....	9
4.3 Camera Integration & Robustness Testing.....	9
4.4 Results and Discussion	9
5 Conclusion.....	Error! Bookmark not defined.
References	11
Appendix A Project Timeline	12

List of figures

	Page
Figure 1: Minigrid 8x8 Environment	5

List of tables

	Page
Table 1: Final Model Parameter Values.....	8

1 Introduction

1.1 Background

Machine Learning is a subdivision of Artificial Intelligence (AI) that uses algorithms to discover patterns in data, enabling a system to “learn” and make predictions or decisions that improve with experience, without explicit programming. Reinforcement Learning (RL) is a subdivision of Machine Learning in which a machine will learn to maximise its cumulative rewards from the feedback it receives through interactivity with its environment.

Deep Neural Networks have proven to be a highly effective way to implement RL algorithms, however, the large memory and computational requirements impeded their deployment onto embedded microcontroller devices. This project will find the most suitable way to condense the Deep Reinforcement Learning (DRL) agents onto those devices.

This report will evaluate the technical feasibility of the project, by explaining the motivation, objectives and planned activities, as well as, expected costs and risks for the study. The project was proposed by Prof HA Engelbrecht, and will be done by Miss AL Beukes as a part of the Mechatronic Skripsie Project 448

1.2 Objectives

The aim of this project is to compress a trained Deep Reinforcement Learning algorithm and deploy the agent onto an embedded device, without trading off the execution speed and performance. The objectives are as follows:

1. Implement Deep Reinforcement Learning algorithm on a desktop computer.
2. Train the Deep Reinforcement Learning agent on images from a virtual environment until a baseline performance is established.
3. Compress the policy network and deploy the agent onto an embedded device.
4. Add a camera to obtain images from the computer screen.

1.3 Motivation

Deep Reinforcement learning is a powerful but resource-intensive for deploying to embedded devices. There is a growing demand for autonomous embedded systems in robotics and IoT devices. It is more feasible and practical for machine learning algorithms to be on-device systems, rather than cloud-dependent.

TinyML is a subfield of machine learning that focuses on optimising AI models to be deployed on resource-constrained microcontroller devices. Previous model compression research primarily focuses on supervised machine learning, rather than compacting reinforcement learning policy networks. The embedded microcontroller unit (MCU) used for this project will be extremely resource-constrained, therefore, deployment of an image-based DRL policy network onto this device will depict a meaningful challenge towards the contribution into the TinyML field.

2 Literature Review

Deep Reinforcement Learning (DRL) has enabled agents to learn effective control policies directly from high-dimensional input, most notably raw pixel observations, by using deep neural networks as function approximators for the action-value function in Reinforcement Learning algorithms. However, this capability comes at a computational cost, which presents a direct obstacle to deploying such agents on resource-constrained embedded platforms. Model compression has emerged as a necessary step in the pipeline connecting a trained DRL agent to a viable embedded implementation.

2.1 Network Pruning and Fine-Tuning Strategies

Network pruning methods are divided into unstructured and structured approaches. Unstructured pruning removes individual weights based on an importance criterion, with minimal accuracy loss, however, requires specialised hardware for implementation. Structured pruning instead removes entire filters, channels, or neurons, producing a smaller network that maps directly onto standard hardware. For embedded deployment, structured pruning is the more practically relevant approach and is the common thread across prior research.

Pruning causes a loss in performance therefore requiring specific strategies to recover the model:

- (i) Policy Distillation: in which an unpruned teacher network guides a pruned student network via supervised imitation of the teacher's action outputs (Xu et al., 2022)
- (ii) Reinforcement-Learning-Based Fine-Tuning, in which the pruned network continues to be updated using RL (Park et al., 2024)

These two strategies slightly mitigate performance losses from network pruning.

2.2 LRP-Based Filter Pruning and Policy Distillation

Xu et al. (2022) proposed a structured pruning method for DQN policy networks that departs from conventional gradient-based importance criteria by instead using Layer-wise Relevance Propagation (LRP), a technique originally developed for explainable AI, to score the importance of each convolutional filter. Relevance scores are propagated backward from the network's output to its input, and each filter's importance is computed as the L1 norm of the relevance assigned to its constituent neurons. Filters are then pruned iteratively in fixed percentage steps, with orphaned connections removed at each step (hard filter pruning).

Policy Distillation is then used as a recovery mechanism, whereby a pre-trained teacher network first interacts with the environment to populate a replay buffer; this buffer is then frozen and reused after every pruning iteration to fine-tune the student network.

The models were evaluated on three Atari benchmarks (Road Runner, Bank Heist, and Pong), the LRP-based criterion consistently achieves higher average reward than weight-magnitude (L1/L2), gradient-based, and Taylor-expansion-based pruning baselines. The paper also reports that pruned models require moderately fewer FLOPs than baseline methods at equivalent pruning rates, attributed to LRP tending to remove filters with a larger number of weights.

Notably, this work stops short of actual embedded deployment, and all results are reported on a GPU workstation.

2.3 Structured Pruning with Scaled Policy Constraints

Park et al. (2024) establishes their importance criterion as a simple L2-norm of channel weights, a deliberately conventional choice that the authors justify by appeal to prior evidence that the criteria remain competitive with more complex alternatives.

However, rather than distillation, pruned policies are recovered using a novel fine-tuning strategy called Scaled Policy Constraint (SPC), followed by structured regularization. The paper traces a lineage from standard Behaviour Cloning (BC), which is mainly used for supervised learning, TD3+BC augments, weighted Policy Constraint (wPC) to the SPC method proposed. SPC replaces this binary weighting of wPC with a continuous ReLU-based weighting of the advantage, allowing the degree of imitation to scale proportionally with action quality rather than being thresholded.

The models were evaluated on the D4RL offline RL benchmark (HalfCheetah, Hopper, Walker2d) and a drone control simulation, SPC is shown to outperform TD3+BC and wPC at multiple structured pruning rates. (Park et al., 2024). This paper addresses the embedded deployment directly with pruned VisionNet policies deployed on an NVIDIA Jetson Xavier NX, which achieved a 2.5-times inference speedup on its Volta GPU and over double the speedup on its Carmel ARM CPU at approximately 87% pruning, with no corresponding drop in cumulative reward on drone control.

3 Progress Completed

This section discusses the project's current progress until the date of submission of this report.

3.1 DRL Agent Development

A deep reinforcement learning agent was implemented and trained before compression of its policy network. This implementation was built from the ground up and the process was divided into sections, which are outlined below.

3.1.1 Tabular Q-Learning Model

The first step was the implementation of a Tabular Q-Learning Model. Because the aim of the project is to compress the policy network, a Q-Learning model was designed. Q-Learning has a value-based policy network, which is easier to compress in comparison to policy-based/actor-critic networks.

The model trained was one used to determine the minimum steps to reaching a goal in a Minigrid environment. An 8x8 grid environment was chosen for its simplicity. The list of actions available for the agent to take in this environment are:

- Turn Left
- Turn Right
- Move Forward

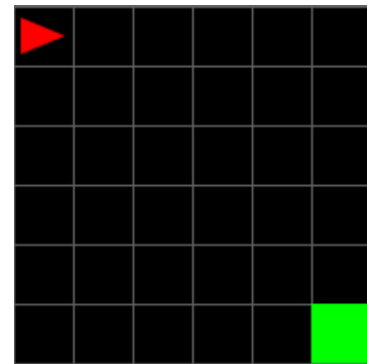


Figure 1: Minigrid 8x8 Environment

To develop a tabular q-learning model:

1. Make the Minigrid Virtual Environment
2. Define an Empty Q-Table
3. Create an Episode Loop to Train the Agent
 - a. Reset the Environment
 - b. Set Current Episode Loss, Reward and Steps to Zero
 - c. Extract the Current State & Hash the State
 - d. Loop though Main Reinforcement Learning Step Loop (4)
 - e. Decay Epsilon
 - f. Log Values to Tensorboard
4. Create the Main Reinforcement Learning Step Loop:
 - a. Perform Epsilon Greedy Exploration:
 - i. Explore the Environment - Picks Random Action, or
 - ii. Exploit the Environment – Select Action for Max Value Function at the Current State
 - b. Extract the Step Information for Taking the Action (i.e. Observation, Reward, Done, Truncated)

- c. Extract the Next State from the Observation & Hash the State
- d. Update the Q-Table using the Bellman Equation
 - i. $E =$
 - ii. $L =$
 - iii. $Q[S, A] =$
- e. Increment Step Count
- f. Calculate the Accumulated Rewards
- g. Move to the Next State
- h. Break Loop
 - i. Goal is Reached (Done), or
 - ii. Max Steps Reached (Truncated)

The model's hyperparameters were then tuned using Optuna to establish the best baseline performance. This process is discussed further in section 3.1.3.

3.1.2 Translation to Deep Q-Network

The Tabular Q-learning algorithm was adapted into a Deep Q-Network by replacing the Q-table with neural network function approximation, which estimates the Q-values directly from the state input.

To create the deep q-network:

1. Make the Minigrid Virtual Environment
2. Define the Policy Network and Target Network (Replaces the Q-Table)
 - a. Instantiate Policy Network - a fully connected Neural Network, which maps states to Q-values for each Action
 - b. Instantiate Target Network - the same architecture
 - c. Copy the Policy Network's Weights to the Target Network
 - d. Ensure the Target Network's Parameters are Static
3. Initialise the Experience Replay Memory
 - a. Define an Experience Structure which Stores: {Current State, Action, Next State, Reward}
 - b. Create a Replay Memory Buffer to Store Past Experience
4. Create an Episode Loop to Train the Agent
 - a. Reset the Environment
 - b. Extract and Preprocess the Current State (normalise & flatten state into an input tensor)
 - c. Loop through the Main Reinforcement Learning Step Loop (5)
 - d. Log Reward and Step Count to Tensorboard
5. Create the Main Reinforcement Learning Step Loop:
 - a. Perform Epsilon-Greedy Exploration:
 - i. Explore the Environment – Pick Random Action, or
 - ii. Exploit the Environment – Select Action for Max Q-value predicted by the Policy Network at the Current State
 - b. Extract the Step Information for Taking the Action

- (i.e. Observation, Reward, Done, Truncated)
- c. Extract and Preprocess the Next State from the Observation (None if the Episode has Ended)
- d. Store the Transition in Experience Replay Memory
- e. Optimise the Policy Network (6)
- f. Move to the Next State
- g. Periodically Update the Target Network
 - i. Copy the Policy Network's Weights at Update Intervals
- h. Break Loop
 - i. Goal is Reached (Done), or
 - ii. Max Steps Reached (Truncated)
- 6. Optimise the Policy Network:
 - a. Sample a Random Experience Mini-Batch from Replay Memory
 - b. Calculate the Predicted Q-Values from the Policy Network
 - c. Calculate the TD-Target estimated by the Target Network
 - i. $E = \text{Estimated Next-State Value}$
 - 1. Max Q-Value from Target Network
 - 2. Zero for Terminal States
 - ii. $\text{TD-Target} = \text{Reward} + (\gamma \times E)$
 - iii. $\text{TD-Error} = \text{TD-Target} - \text{Predicted Q-Value}$
 - d. Calculate the Loss between Predicted Q-Values and TD-Targets (Mean-Squared Error)
 - e. Perform Gradient Descent to Minimise the Loss
 - i. Clamp Gradients to the Range $[-1, 1]$ for stability
 - ii. Log Loss and TD-Error to Tensorboard

Similarly to the Tabular Q-Learning Model, the hyperparameters were tuned using Optuna to establish the best baseline performance.

3.1.3 DQN Model Architecture and Optimization

To establish a strong foundational baseline performance model, hyperparameter tuning was implemented.

Optuna is an open-source software, which can be implemented directly with Pytorch, for automated hyperparameter optimization search. It is easily integrated into machine learning frameworks through 3 simple steps:

1. Define an Objective Function
 - a. Trial Hyperparameter Values within Suggested Range
2. Create a Study and Invoke the Optimize Method -for a number of trials

Incorporating these steps into the model optimizes model performance. The final model's parameters were:

Table 1: Final Model Parameter Values

Parameter	Final Model Value
alpha	0.5460858047587036
gamma	0.9976442756947885
epsilon_start	0.5412795304262256
epsilon_decay	0.9936930717790172
epsilon_min	0.045001959694373365

3.2 Policy Network Compression

The final Deep Q-Learning Model’s policy network will be compressed onto a resource-intensive embedded device, specifically the Raspberry Pi 2 Pico W. This device has a storage capacity of 520KB on chip SRAM. The policy network will have to be sufficiently compressed to be deployed onto this device.

3.2.1 Research Network Compression Methods

There has been an ongoing investigation into previous research papers associated with the topic of compression of DRL agents onto embedded devices. A literature review was conducted with the current information gathered as of the date of this report. Further investigation and implementation of these strategies, along with the timeline for the remainder of this report is outlined in the next section.

Further reading includes Quantization in Deep Reinforcement Learning and the existing research in the TinyML field.

4 Planned Activities

All planned activities is divided and scheduled for completion according to the rough timeline in Appendix A.

4.1 Policy Network Compression

Currently, research and implementation of multiple policy network compression methods are being tested and evaluated. Once an optimal compression method is developed, the policy network will be compressed and ready for deployment onto the embedded microcontroller.

The performance of all the different compression methods will be tested and documented for further discussion.

4.2 Embedded Deployment

A Raspberry Pi 2 Pico W has been ordered through the Electrical and Electronic Engineering Department. Once obtained and the compressed module is ready for deployment, the policy network will be installed onto the device.

There will be several tests conducted to ensure the performance of the network works on the device and those metrics will be documented for evaluation.

Once the deployment process is under way, a compatible camera module will be ordered through the department.

4.3 Camera Integration & Robustness Testing

With the embedded device up and running, once received, a camera module will be integrated to the microcontroller. Further tests will then be conducted on the agent, to account for real-world noise, latency and other imperfections.

4.4 Results and Discussion

Finally, the various agent performance results will be compiled, evaluated and discussed in the final report document. Whereby an optimal compression strategy for deploying reinforcement learning algorithms onto resource-intensive embedded devices should be determined.

5 Conclusion

This report has outlined the progress made towards the compression of a Deep Reinforcement Learning agent for deployment onto an embedded microcontroller. A working DQN agent was developed and trained, progressing from a tabular Q-learning baseline to a neural network-based policy, with hyperparameters optimised using Optuna. A literature review of existing pruning and distillation strategies has provided insights into previous compression approaches. The remaining work involves, further research and then compressing the policy network, deploying it to the Raspberry Pi Pico 2 W, and integrating a camera for robustness testing.

References

Park, S., Kim, H., Kim, H. and Choi, J. (2024) '*Pruning with scaled policy constraints for light-weight reinforcement learning*', IEEE Access, 12, pp. 36055–36065.

Xu, R., Luan, S., Gu, Z., Zhao, Q. and Chen, G. (2022) '*LRP-based policy pruning and distillation of reinforcement learning agents for embedded systems*', in 2022 IEEE 25th International Symposium On Real-Time Distributed Computing (ISORC). IEEE.

Appendix A Project Timeline

This Appendix structured the remaining project Planned Activities into a timeline for the remainder of the project life cycle.

Milestone	Target Date	Status
Investigate Policy Network Compression Strategies	7 August 2026	In Progress
Evaluate Final Model's Performance	7 August 2026	In Progress
Implement Policy Network Compression Strategies	14 August 2026	Not Started
Evaluate Compression Performance	21 August 2026	Not Started
Deploy Compressed Policy Network onto Embedded Device	28 August 2026	Not Started
Improve Policy as Necessary	11 September 2026	Not Started
Test and Evaluate Model's Performance	18 September 2026	Not Started
Add Camera and Test for Robustness	25 September 2026	Not Started
Final Report Discussion	16 October 2026	In Progress